# Experiment Title: Searching Files and Content in Linux Terminal

**Topic:** Locating Files with `find` and Searching Text Content with `grep`

## 1. Objective

The objective of this lab is to learn how to search for files and text efficiently in Linux systems. By completing this experiment, we aim to:

- Search and locate files by name, pattern, and file type using the **`find`** command.
- Search for specific words and patterns inside text files using **`grep`**.
- Master essential **`grep`** flags: case-insensitive search (**`-i`**), line numbers (**`-n`**), and inverted match (**`-v`**).
- Recursively search across entire folder structures using **`grep -r`** and **`grep -rn`**.
- Understand when to use **`find`** (finding files on disk) versus **`grep`** (finding text inside files).

## 2. Equipment / Software Requirements

| Item | Details | Item | Details |
|---|---|---|---|
| **Operating System** | Linux (NixOS 26.11 / Ubuntu 24.04) | **Host Machine** | `firefly` |
| **User Account** | `ratul` | **Command Shell** | GNU Bash 5.2 |
| **Terminal Emulator** | Linux Terminal (Kitty / Ghostty) | **Search Tools** | GNU Findutils (`find`), GNU Grep (`grep`) |

## 3. Theoretical Background

When working on large Linux systems with thousands of files, finding the right file or the right line of code manually is impossible. Linux provides two complementary search tools: **`find`** searches through the directory tree looking for *files and folders*, while **`grep`** searches *inside the text contents* of those files.

### 3.1 Commands Practiced in This Lab

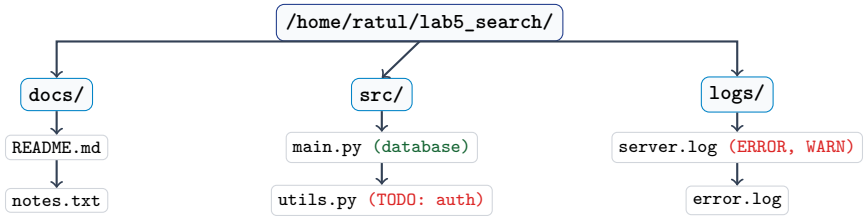
| Command | Syntax | Simple Meaning / Purpose | Example |
|---|---|---|---|
| `find` | `find <dir> -name "<pattern>"` | Finds files matching a name pattern. | `find . -name "*.log"` |
| `find -iname` | `find <dir> -iname "<pattern>"` | Case-insensitive search for file names. | `find . -iname "notes*"` |
| `find -type` | `find <dir> -type <f\|d>` | Filters results by type: `f` (file) or `d` (directory). | `find . -type d` |
| `grep` | `grep "<text>" <file>` | Searches for exact text inside a specified file. | `grep "ERROR" server.log` |
| `grep -i` | `grep -i "<text>" <file>` | Ignores uppercase/lowercase differences. | `grep -i "error" server.log` |
| `grep -n` | `grep -n "<text>" <file>` | Displays the line number where match was found. | `grep -n "fail" server.log` |
| `grep -v` | `grep -v "<text>" <file>` | Inverts match: shows lines that do *not* match. | `grep -v "DEBUG" app.log` |
| `grep -r` | `grep -r "<text>" <dir>` | Recursively searches all files inside folders. | `grep -r "database" .` |
| `grep -rn` | `grep -rn "<text>" <dir>` | Recursive search showing file names and line numbers. | `grep -rn "TODO" .` |

### 3.2 Key Differences: Find vs. Grep

| Feature | `find` | `grep` |
|---|---|---|
| **Primary Goal** | Locating files and directories on disk. | Locating lines of text inside files. |
| **Search Target** | File names, file sizes, permissions, modification dates. | Words, phrases, patterns inside file content. |
| **Analogy** | Finding a specific book in a large library. | Finding a specific sentence inside a book. |

## 4. Equipment / Demo Directory Setup

To practice search operations, we set up a mock project workspace called **lab5_search** inside **/home/ratul**. It contains source code, configuration files, notes, and log files:




## 5. Procedure / Steps

We practiced file and content searching using the following organized steps:

1. Initialized our practice directory **lab5_search** and navigated inside.
2. Used **find** to locate all **.log** files anywhere in the project tree.
3. Searched case-insensitively for documentation files using **find . -iname "readme*"**.
4. Filtered search output by item type: directories only (**find . -type d**) and regular files only (**find . -type f**).
5. Searched for exact occurrences of **"ERROR"** inside **logs/server.log** using **grep**.
6. Performed case-insensitive text search using **grep -i** and printed line numbers with **grep -n**.
7. Inverted the search match using **grep -v** to filter out noisy routine messages.
8. Recursively searched all folders for configuration variables (**grep -r**) and developer comments (**grep -rn**).

## 6. Result / Output Demonstration

All commands were executed under user account **ratul** on host machine **firefly**.

### 6.1 Session 1: Finding Files by Name, Extension, and Type with find

ratul@firefly: ~/lab5_search — bash

```
ratul@firefly:~/lab5_search$ find . -name "*.log"
./logs/error.log
./logs/server.log
ratul@firefly:~/lab5_search$ find . -iname "readme*"
./docs/README.md
ratul@firefly:~/lab5_search$ find . -type d
.
./docs
./src
./logs
ratul@firefly:~/lab5_search$ find . -type f -name "*.py"
./src/main.py
./src/utils.py
ratul@firefly:~/lab5_search$ _
```

**Explanation of Session 1 Results:**

- **find . -name "*.log"**: Walked through all subdirectories and printed paths of all log files. Quotes around "*.log" prevent the shell from expanding the wildcard prematurely.
- **find . -iname "readme*"**: Found README.md even though we typed the pattern in lowercase.
- **find . -type d**: Restricted the output strictly to folders, ignoring regular files.
- **find . -type f -name "*.py"**: Combined the type filter (**-type f**) with an extension filter (**-name "*.py"**) to locate Python scripts.

### 6.2 Session 2: Searching File Content with `grep`, `grep -r`, and Flags

```
ratul@firefly: ~/lab5_search — bash

ratul@firefly:~/lab5_search$ grep "ERROR" logs/server.log
[09:15:20] ERROR Database connection pool exhausted on port 5432
[09:22:45] ERROR Worker node 3 failed health check

ratul@firefly:~/lab5_search$ grep -i "warning" logs/server.log
[09:10:02] WARN  High memory threshold exceeded: 82%
[09:12:30] Warning: Slow query response time (420ms)

ratul@firefly:~/lab5_search$ grep -n "failed" logs/server.log
24:[09:22:45] ERROR Worker node 3 failed health check

ratul@firefly:~/lab5_search$ grep -v "INFO" logs/server.log | head -n 2
[09:10:02] WARN High memory threshold exceeded: 82%
[09:15:20] ERROR Database connection pool exhausted on port 5432

ratul@firefly:~/lab5_search$ grep -r "database" .
src/main.py:DB_HOST = "database.internal.net"
docs/notes.txt:Remember to run database migrations before deploy

ratul@firefly:~/lab5_search$ grep -rn "TODO" .
src/utils.py:14:# TODO: add token authentication
docs/notes.txt:3:- TODO: verify backup cron job

ratul@firefly:~/lab5_search$ _
```

**Explanation of Session 2 Results:**

- `grep "ERROR"`: Scanned `logs/server.log` and printed only lines containing the exact uppercase word `ERROR`.
- `grep -i`: Ignored case distinctions, successfully matching both `WARN` and `Warning`.
- `grep -n`: Included line number (`24:`) alongside the matched text, allowing quick navigation in editors.
- `grep -v`: Inverted the search to exclude standard routine `INFO` messages, highlighting only important warnings and errors.
- `grep -r`: Dived into every subfolder (`src/`, `docs/`, `logs/`) and printed the filename and matching line for `database`.
- `grep -rn`: Combined recursion and line numbering, making it easy for developers to find code tasks (`TODO`) across a codebase.

## 7. Conclusion

In this lab, we mastered essential search utilities in the Linux terminal:

1. We used **`find`** to search the filesystem tree for files and directories based on patterns (**`-name`**), case-insensitivity (**`-iname`**), and types (**`-type f`**, **`-type d`**).
2. We used **`grep`** to scan inside text files, leveraging flags like **`-i`** (ignore case), **`-n`** (show line numbers), and **`-v`** (invert match).
3. We applied recursive search using **`grep -r`** and **`grep -rn`** to rapidly discover configuration strings and code comments across complex directory structures.

Together, **`find`** and **`grep`** provide a fast, reliable, and powerful toolkit for navigating and troubleshooting Linux environments.